

Data Mining

Association Analysis: Basic Concepts and Algorithms

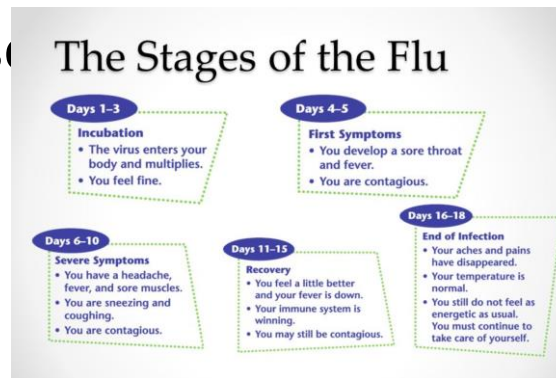
What Are Patterns?

□ What are patterns?

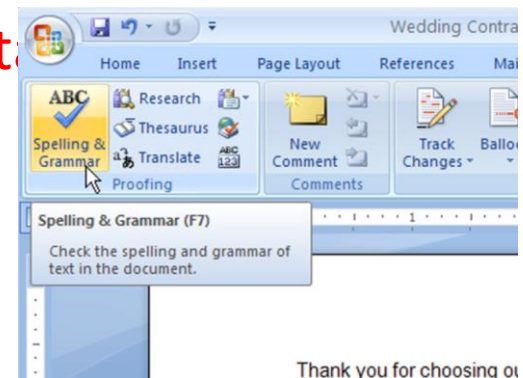
- **Patterns:** A set of items, subsequences, or substructures that occur frequently together (or strongly correlated) in a data set



Frequent item set



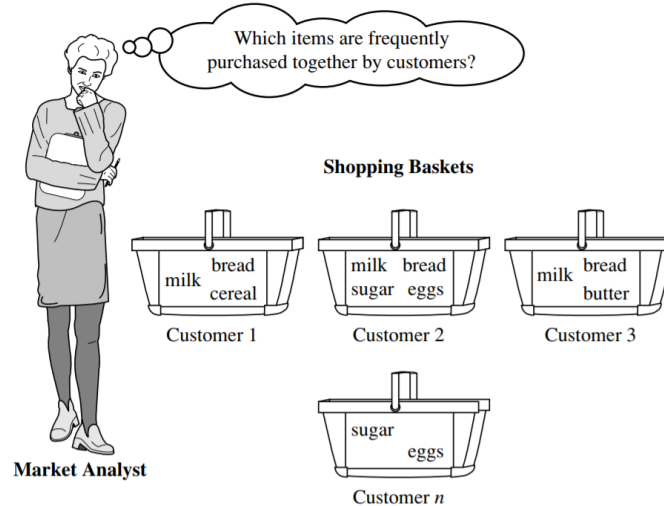
Frequent sequences



Frequent structures

What Is Pattern Discovery?

- **Pattern discovery:** Uncovering patterns from massive data sets
- It can answer questions such as:
 - What products were often purchased together?
 - What are the subsequent purchases after buying an iPad?



Association Rule Mining

- Given a set of transactions, find rules that will predict the occurrence of an item based on the occurrences of other items in the transaction
- An association rule consists of two parts, i.e., an antecedent (if) and a consequent (then).

Market-Basket transactions

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Example of Association Rules

$\{\text{Diaper}\} \rightarrow \{\text{Beer}\},$
 $\{\text{Milk, Bread}\} \rightarrow \{\text{Eggs, Coke}\},$
 $\{\text{Beer, Bread}\} \rightarrow \{\text{Milk}\},$

Implication means co-occurrence,
not causality!

Basic Concepts: Transactional Database

□ Transactional Database (TDB)

- Each transaction is associated with an identifier, called a TID.
- May also have counts associated with each item sold

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

Transaction data: a set of documents

- **A text document data set. Each document is treated as a “bag” of keywords**

doc1: Student, Teach, School
doc2: Student, School
doc3: Teach, School, City, Game
doc4: Baseball, Basketball
doc5: Basketball, Player, Spectator
doc6: Baseball, Coach, Game, Team
doc7: Basketball, Team, City, Game

Definition: Frequent Itemset

□ Itemset

- A collection of one or more items
 - ◆ Example: {Milk, Bread, Diaper}
- k-itemset
 - ◆ An itemset that contains k items

□ Support count (σ)

- Frequency of occurrence of an itemset
- E.g. $\sigma(\{\text{Milk, Bread, Diaper}\}) = 2$

□ Relative support

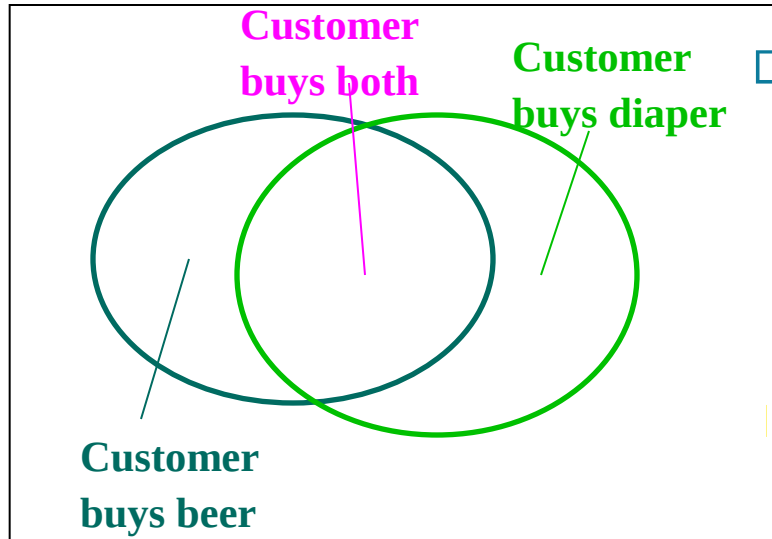
- The fraction of transactions that contains X (i.e., the **probability** that a transaction contains X)
- E.g. $s(\{\text{Milk, Bread, Diaper}\}) = 2/5$

□ Frequent Itemset

- An itemset whose support is greater than or equal to a *minsup* threshold

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Rule Measures: Support and Confidence



Find all the rules $X \& Y \Rightarrow Z$ with minimum confidence and support

- support, s , probability that a transaction contains $\{X \cup Y \cup Z\}$
- confidence, c , conditional probability that a transaction having $\{X \cup Y\}$ also contains Z

Transaction ID	Items Bought
2000	A,B,C
1000	A,C
4000	A,D
5000	B,E,F

Let minimum support 50%, and minimum confidence 50%, we have

- $A \Rightarrow C$ (50%, 66.6%)
- $C \Rightarrow A$ (50%, 100%)

Definition: Association Rule

□ Association Rule

- An implication expression of the form $X \rightarrow Y$, where X and Y are itemsets
- Example:
 $\{\text{Milk, Diaper}\} \rightarrow \{\text{Beer}\}$

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

□ Rule Evaluation Metrics

- Support (s)
 - ◆ Fraction of transactions that contain both X and Y
- Confidence (c)

Example:

$\{\text{Milk, Diaper}\} \Rightarrow \text{Beer}$

◆ Measures how often items in Y appear in transactions that contain X

$$s = \frac{\sigma(\text{Milk, Diaper, Beer})}{|T|} = \frac{2}{5} = 0.4$$

◆ Confidence is the conditional probability of Y when X has already occurred

$$c = \frac{\sigma(\text{Milk, Diaper, Beer})}{\sigma(\text{Milk, Diaper})} = \frac{2}{3} = 0.67$$

Mining Association Rules—An Example

Transaction ID	Items Bought
2000	A,B,C
1000	A,C
4000	A,D
5000	B,E,F

Min. support 50%
Min. confidence 50%

Frequent Itemset	Support
{A}	75%
{B}	50%
{C}	50%
{A,C}	50%

For rule $A \Rightarrow C$:

support = $P(A \cup C) = 50\%$

confidence = $P(C|A) = 66.6\%$

Association Rule Mining Task

- Given a set of transactions T , the goal of association rule mining is to find all rules having
 - support $\geq \textit{minsup}$ threshold
 - confidence $\geq \textit{minconf}$ threshold
 - Brute-force approach:
 - List all possible association rules
 - Compute the support and confidence for each rule
 - Prune rules that fail the *minsup* and *minconf* thresholds
- ⇒ **Computationally prohibitive!**

Mining Association Rules

TID	Items
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Example of Rules:

$\{\text{Milk, Diaper}\} \rightarrow \{\text{Beer}\}$ ($s=0.4, c=0.67$)
 $\{\text{Milk, Beer}\} \rightarrow \{\text{Diaper}\}$ ($s=0.4, c=1.0$)
 $\{\text{Diaper, Beer}\} \rightarrow \{\text{Milk}\}$ ($s=0.4, c=0.67$)
 $\{\text{Beer}\} \rightarrow \{\text{Milk, Diaper}\}$ ($s=0.4, c=0.67$)
 $\{\text{Diaper}\} \rightarrow \{\text{Milk, Beer}\}$ ($s=0.4, c=0.5$)
 $\{\text{Milk}\} \rightarrow \{\text{Diaper, Beer}\}$ ($s=0.4, c=0.5$)

Observations:

- All the above rules are binary partitions of the same itemset:
 $\{\text{Milk, Diaper, Beer}\}$
- Rules originating from the same itemset have identical support but can have different confidence
- Thus, we may decouple the support and confidence requirements

Mining Association Rules

- Two-step approach:

1. Frequent Itemset Generation

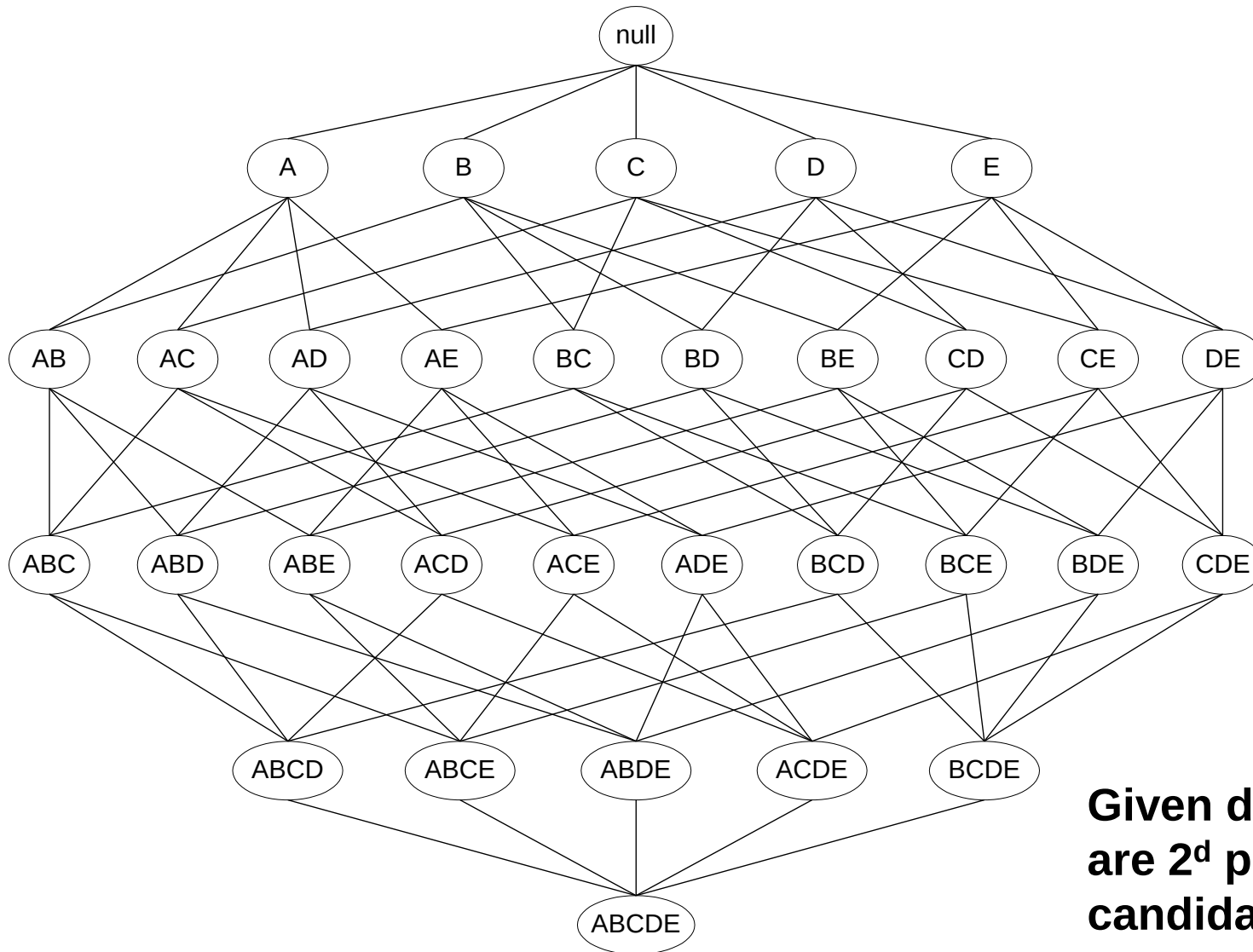
- Generate all itemsets whose support $\geq$ minsup

2. Rule Generation

- Generate high confidence rules from each frequent itemset, where each rule is a binary partitioning of a frequent itemset

- Frequent itemset generation is still computationally expensive

Frequent Itemset Generation

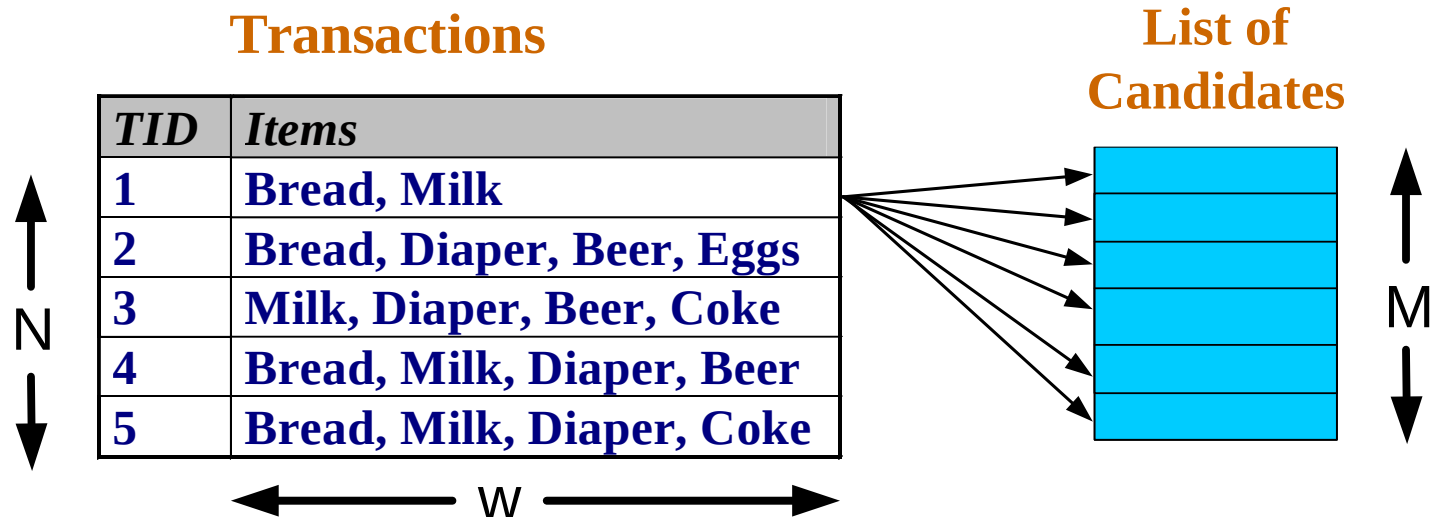


Given d items, there are 2^d possible candidate itemsets

Frequent Itemset Generation

□ Brute-force approach:

- Each itemset in the lattice is a **candidate** frequent itemset
- Count the support of each candidate by scanning the database



- Match each transaction against every candidate
- Complexity $\sim O(NMw) \Rightarrow$ **Expensive since $M = 2^d$!!!**

Reducing Number of Candidates

□ Apriori principle:

- If an itemset is frequent, then all of its subsets must also be frequent
- uses prior knowledge of frequent itemset properties.

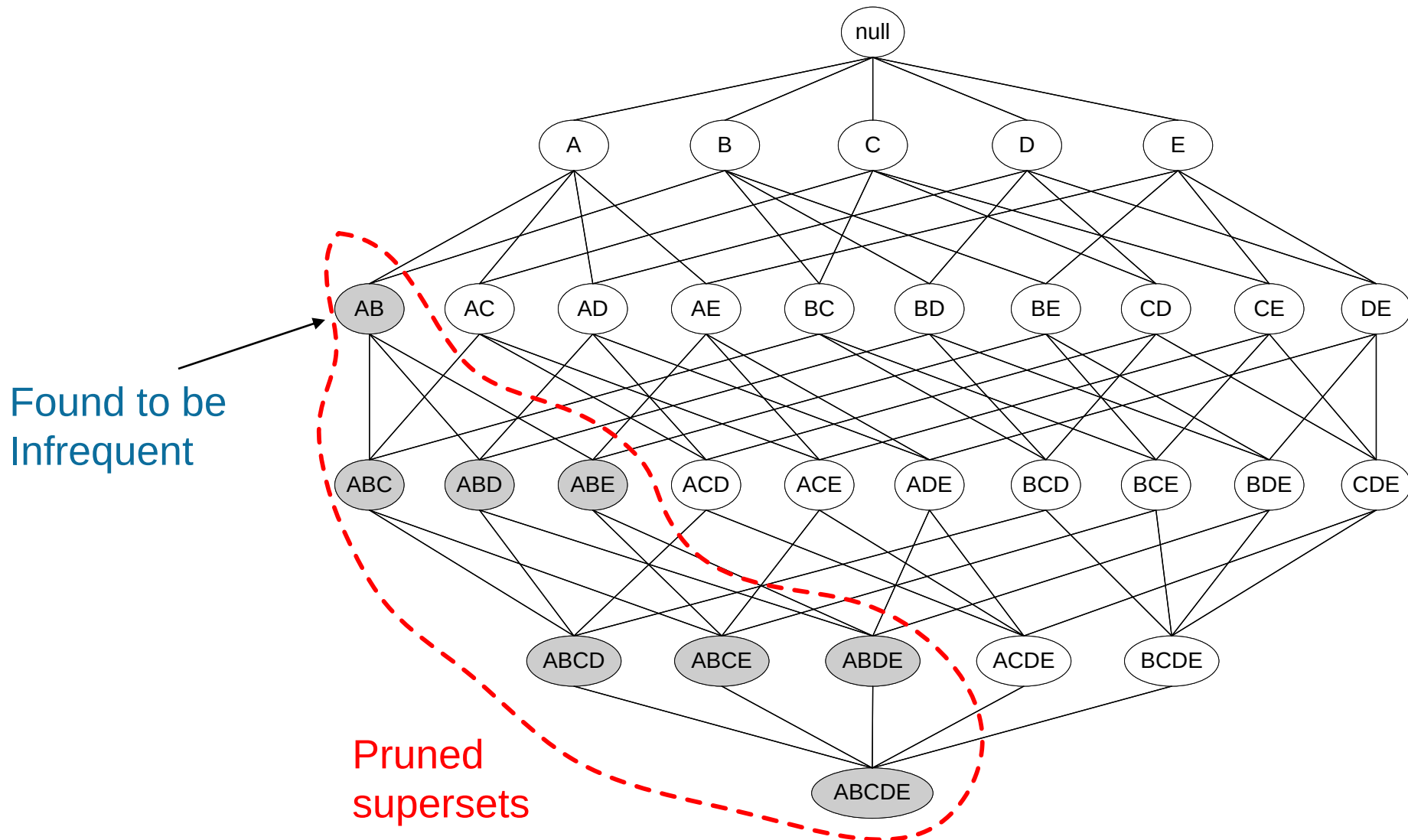
□ Apriori principle holds due to the following property of the support measure:

$$\forall X, Y : (X \subseteq Y) \Rightarrow s(X) \geq s(Y)$$

- Support of an itemset never exceeds the support of its subsets
- This is known as the **anti-monotone** property of support

Illustrating Apriori Principle

Lattice structure of frequent itemsets



Illustrating Apriori Principle

Item	Count
Bread	4
Coke	2
Milk	4
Beer	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset	Count
{Bread,Milk}	3
{Bread,Beer}	2
{Bread,Diaper}	3
{Milk,Beer}	2
{Milk,Diaper}	3
{Beer,Diaper}	3

Pairs (2-itemsets)

(No need to generate candidates involving Coke or Eggs)



Itemset	Count
{Bread,Milk,Diaper}	3

Triplets (3-itemsets)

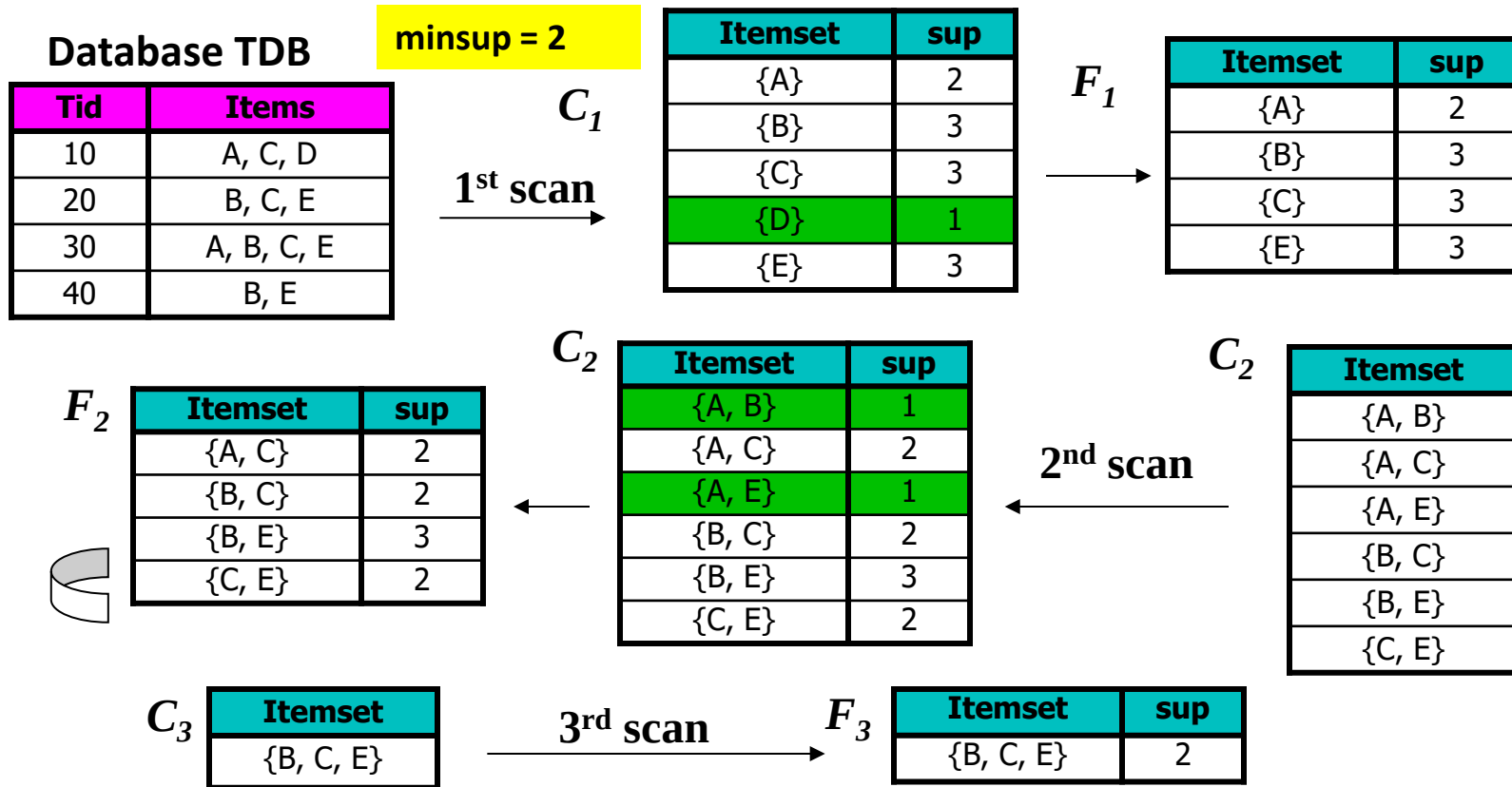
Minimum Support = 3



Mining Frequent Itemsets: the Key Step

1. Find the *frequent itemsets*: the sets of items that have minimum support
 - A subset of a frequent itemset must also be a frequent itemset
 - ◆ i.e., if $\{AB\}$ is a frequent itemset, both $\{A\}$ and $\{B\}$ should be a frequent itemset
 - Iteratively find frequent itemsets with cardinality from 1 to k (k -itemset)
2. Use the frequent itemsets to generate association rules.

The Apriori Algorithm—An Example



Apriori Algorithm

□ Method:

- Let $k=1$
- Generate frequent itemsets of length 1
- Repeat until no new frequent itemsets are identified
 - ◆ Generate length $(k+1)$ candidate itemsets from length k frequent itemsets
 - ◆ Prune candidate itemsets containing subsets of length k that are infrequent
 - ◆ Count the support of each candidate by scanning the DB
 - ◆ Eliminate candidates that are infrequent, leaving only those that are frequent

The Apriori Algorithm

- **Join Step:** C_k is generated by joining L_{k-1} with itself
- **Prune Step:** Any $(k-1)$ -itemset that is not frequent cannot be a subset of a frequent k -itemset

- Pseudo-code:

C_k : Candidate itemset of size k

L_k : frequent itemset of size k

$L_1 = \{\text{frequent items}\};$

for ($k = 1; L_k \neq \emptyset; k++$) **do begin**

C_{k+1} = candidates generated from L_k ;

for each transaction t in database **do**

 increment the count of all candidates in C_{k+1}
 that are contained in t

L_{k+1} = candidates in C_{k+1} with min_support

end

return $\cup_k L_k$;